

From Data to Solutions



Modern reinforcement learning

Learning to decide, from trial and error

Carlos Cotrini



Section 1

What reinforcement learning is

1

What reinforcement learning is

A program learns to make a **sequence of decisions** by trying them and keeping what worked.

Nobody writes the rules. It is told only **what a good outcome is worth**.

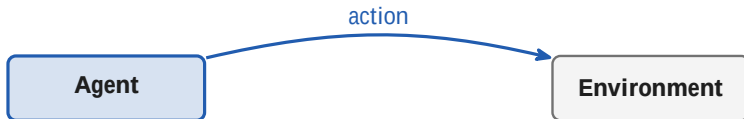
Agent

Environment

What reinforcement learning is

A program learns to make a **sequence of decisions** by trying them and keeping what worked.

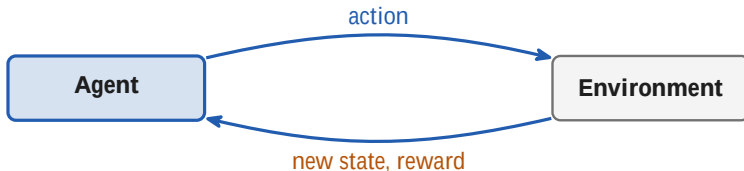
Nobody writes the rules. It is told only **what a good outcome is worth**.



What reinforcement learning is

A program learns to make a **sequence of decisions** by trying them and keeping what worked.

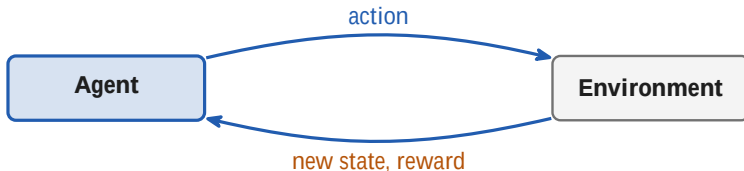
Nobody writes the rules. It is told only **what a good outcome is worth**.



What reinforcement learning is

A program learns to make a **sequence of decisions** by trying them and keeping what worked.

Nobody writes the rules. It is told only **what a good outcome is worth**.



Try, observe, keep what worked



Section 2

Where it is used

2

Where it is used today

Where it is used today

Language models
which answer to give



Where it is used today

Language models

which answer to give



Data centres

how hard to cool



Where it is used today

Language models

which answer to give



Data centres

how hard to cool



Robotics

how to grasp and move



Where it is used today

Language models

which answer to give



Data centres

how hard to cool



Robotics

how to grasp and move



Logistics

what to stock, when



Where it is used today

Language models

which answer to give



Data centres

how hard to cool



Robotics

how to grasp and move



Logistics

what to stock, when



Recommendation

what to show next



Where it is used today

Language models

which answer to give



Data centres

how hard to cool



Robotics

how to grasp and move



Logistics

what to stock, when



Recommendation

what to show next



Games

which move to play



Where it is used today

Language models

which answer to give



Data centres

how hard to cool



Robotics

how to grasp and move



Logistics

what to stock, when



Recommendation

what to show next



Games

which move to play



One decision, then another, then the bill



Section 3

Markov decision processes

3

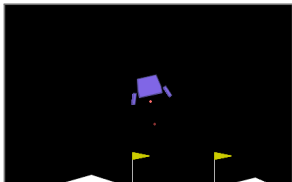
Markov decision process: landing a spacecraft

States

Actions

Transitions

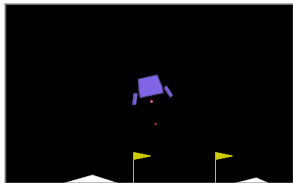
Rewards



position, speed, tilt, legs

Markov decision process: landing a spacecraft

States



position, speed, tilt, legs

Actions

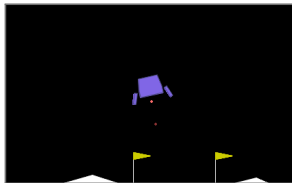
- 0 do nothing
- 1 left engine
- 2 main engine
- 3 right engine

Transitions

Rewards

Markov decision process: landing a spacecraft

States

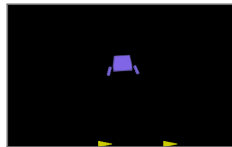


position, speed, tilt, legs

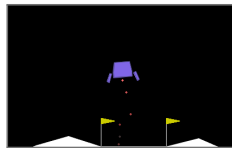
Actions

- 0 do nothing
- 1 left engine
- 2 main engine
- 3 right engine

Transitions



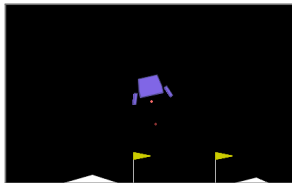
↓ main engine



Rewards

Markov decision process: landing a spacecraft

States

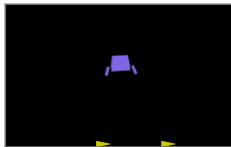


position, speed, tilt, legs

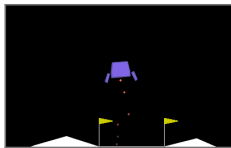
Actions

- 0 do nothing
- 1 left engine
- 2 main engine
- 3 right engine

Transitions



↓ main engine

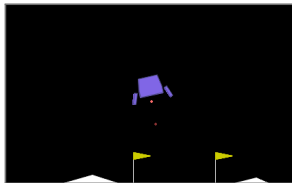


Rewards

- +100 landed
- 100 crashed
- +10 leg down
- 0.3 main engine
- small each frame for being nearer, slower, more upright

Markov decision process: landing a spacecraft

States

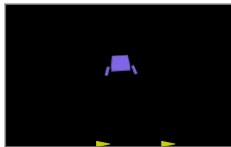


position, speed, tilt, legs

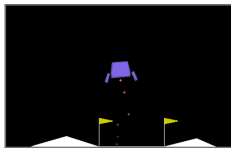
Actions

- 0 do nothing
- 1 left engine
- 2 main engine
- 3 right engine

Transitions



↓ main engine



Rewards

- +100 landed
- 100 crashed
- +10 leg down
- 0.3 main engine
- small each frame for being nearer, slower, more upright

Fly it yourself: fdd-hs26.github.io/projects/fly-the-lander/

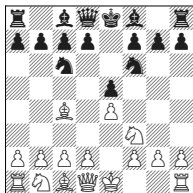
Markov decision process: chess

States

Actions

Transitions

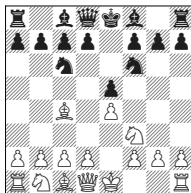
Rewards



where every piece stands

Markov decision process: chess

States



where every piece stands

Actions

Nf3 a knight

Bc4 a bishop

O-O castling

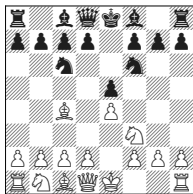
every legal move

Transitions

Rewards

Markov decision process: chess

States



where every piece stands

Actions

- Nf3** a knight
 - Bc4** a bishop
 - O-O** castling
- every legal move

Transitions



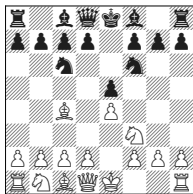
↓ castles



Rewards

Markov decision process: chess

States



where every piece stands

Actions

- Nf3** a knight
 - Bc4** a bishop
 - O-O** castling
- every legal move

Transitions



↓ castles

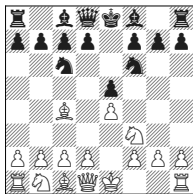


Rewards

- +1** checkmate
- 1** checkmated
- 0** draw
- 0** every other move

Markov decision process: chess

States



where every piece stands

Actions

- Nf3** a knight
 - Bc4** a bishop
 - O-O** castling
- every legal move

Transitions



↓ castles



Rewards

- +1** checkmate
- 1** checkmated
- 0** draw
- 0** every other move

The reward can arrive only at the very end

Markov decision process: what to stock

States



9 left

Actions

Transitions

Rewards

Markov decision process: what to stock

States



Actions

- 0 order nothing
- + order a little
- ++ order a lot

Transitions

Rewards

Markov decision process: what to stock

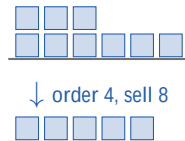
States



Actions

- 0 order nothing
- + order a little
- ++ order a lot

Transitions



Rewards

Markov decision process: what to stock

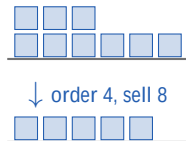
States



Actions

- 0 order nothing
- + order a little
- ++ order a lot

Transitions



Rewards

- + margin on a sale
- lost sale
- holding cost
- writing off

Markov decision process: what to stock

States

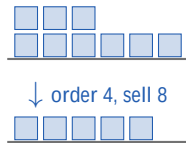


9 left

Actions

- 0 order nothing
- + order a little
- ++ order a lot

Transitions



Rewards

- + margin on a sale
- lost sale
- holding cost
- writing off

Same four parts, no spacecraft required



Section 4

Policies and episodes

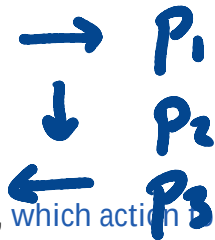
4

The policy

Any procedure that decides, in every state, **which action to take**.

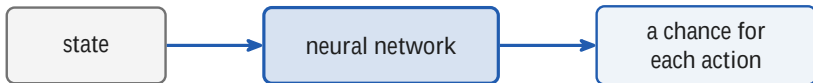
$$\pi : S \rightarrow A$$

The policy



Any procedure that decides, in every state, which action to take.

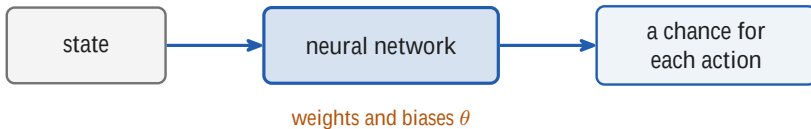
$$\pi : S \rightarrow A$$



The policy

Any procedure that decides, in every state, **which action to take**.

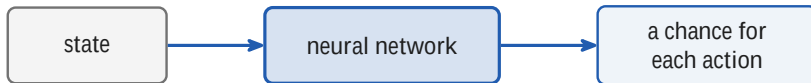
$$\pi : S \rightarrow A$$



The policy

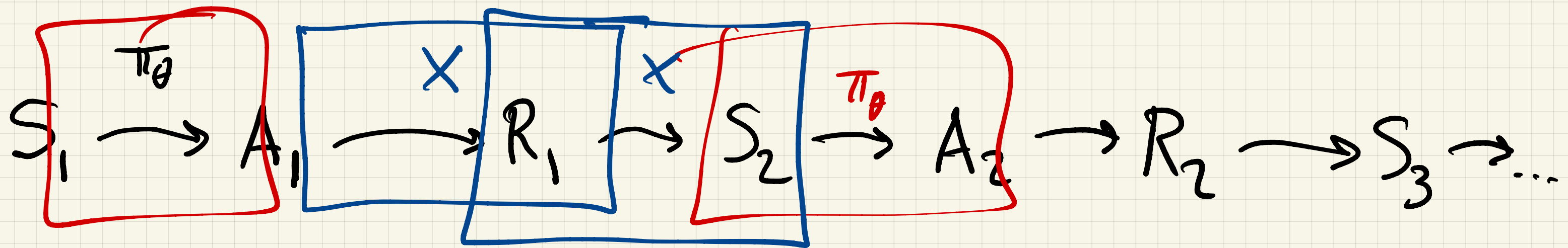
Any procedure that decides, in every state, **which action to take**.

$$\pi : S \rightarrow A$$



weights and biases θ

$$\pi_{\theta}$$



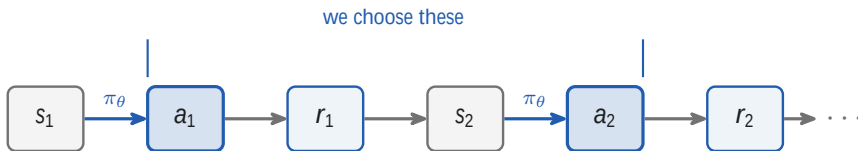
$$G = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots + \gamma^t R_t + \dots$$

$$0 < \gamma < 1$$

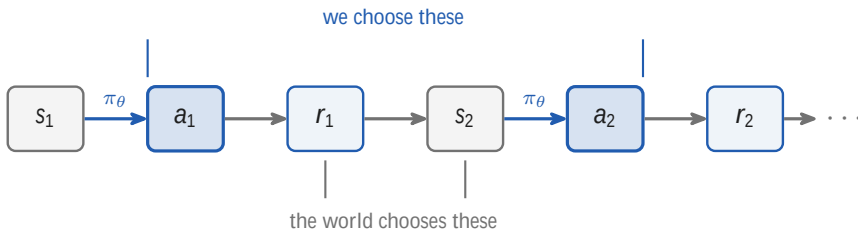
One episode



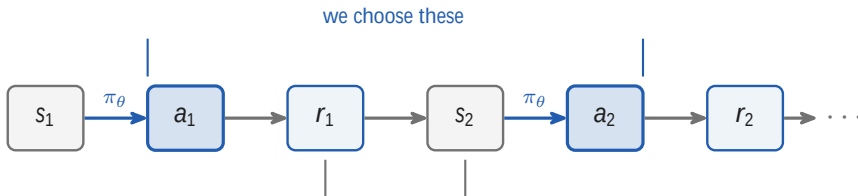
One episode



One episode



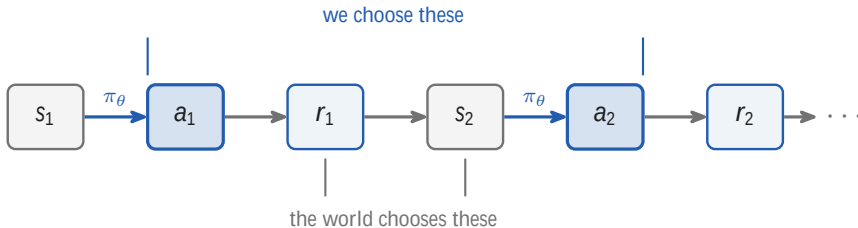
One episode



the world chooses these

$$\mathbb{E}[G_t] = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$
$$\mathbb{E}[G_1] = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$$

One episode



$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

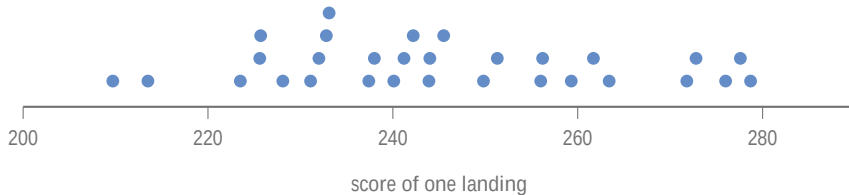


Section 5

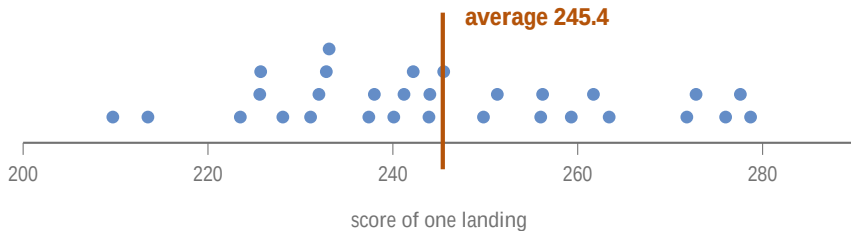
The objective

5

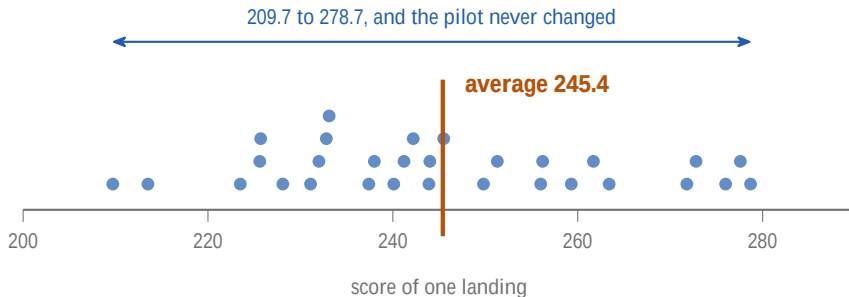
The same pilot, thirty landings



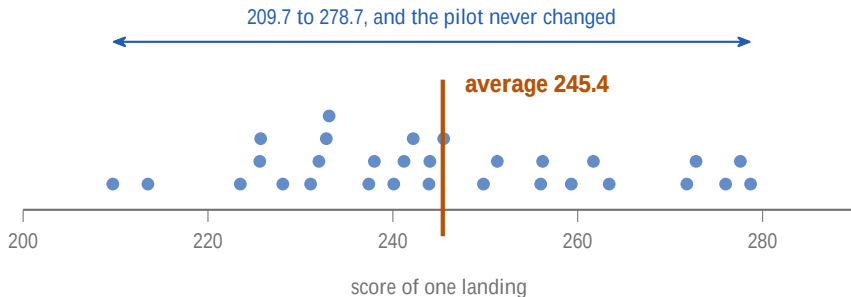
The same pilot, thirty landings



The same pilot, thirty landings

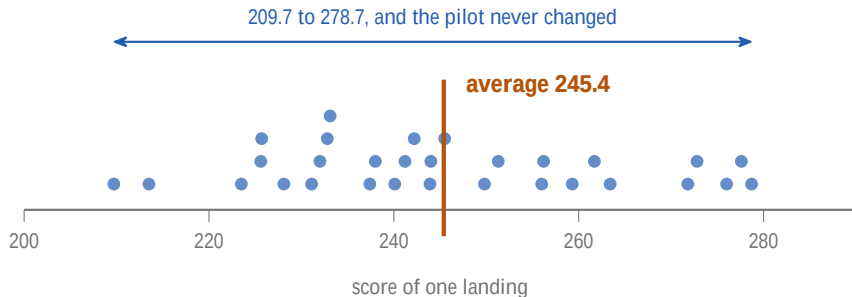


The same pilot, thirty landings



G is a property of **one attempt** $\mathbb{E}[G]$ is a property of **the pilot**

The same pilot, thirty landings



G is a property of **one attempt** $\mathbb{E}[G]$ is a property of **the pilot**

You cannot improve a dot. You can move the line.

© The objective

The goal



$\leftarrow p_1$
 $\downarrow p_2$
 $\rightarrow p_3$

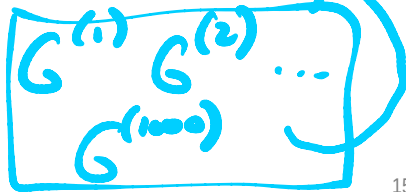
Find the weights θ

$\text{Avg} = J(\theta)$

that maximize $E[G_\theta]$



π_θ



$J(\theta)$ $\rightarrow$

$$\nabla J(\theta) = \frac{1}{T} \sum_{t \leq T} G_t \nabla \log \pi(a_t | s_t)$$

The goal

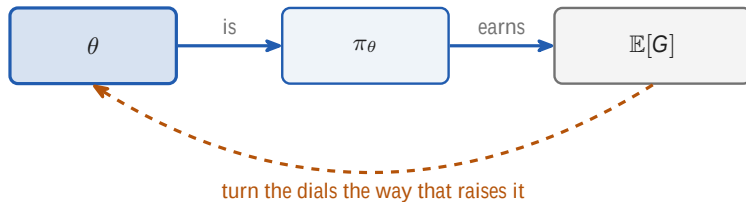
Find the weights θ that make π_θ as good as possible

The goal

Find the weights θ that make $\mathbb{E}[G]$ as large as possible

The goal

Find the weights θ that make $\mathbb{E}[G]$ as large as possible





Section 6

Finding the weights

6

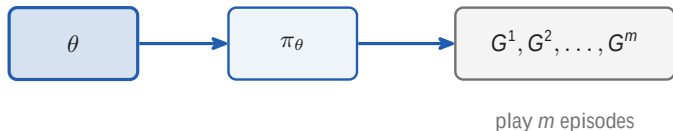
We have played this game already

<https://fdd-hs26.github.io/projects/grasshopper/>



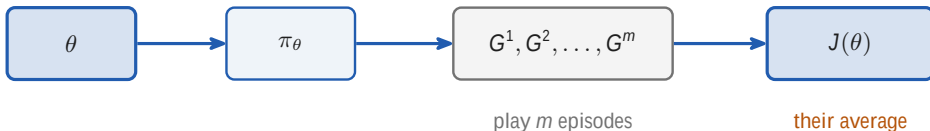
We have played this game already

<https://fdd-hs26.github.io/projects/grasshopper/>



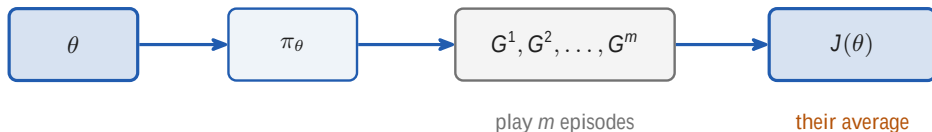
We have played this game already

<https://fdd-hs26.github.io/projects/grasshopper/>



We have played this game already

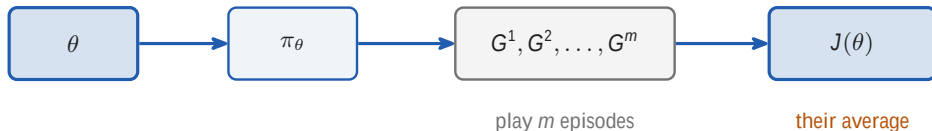
<https://fdd-hs26.github.io/projects/grasshopper/>



$J(\theta)$ is our estimate of $\mathbb{E}[G]$

We have played this game already

<https://fdd-hs26.github.io/projects/grasshopper/>



$J(\theta)$ is our estimate of $\mathbb{E}[G]$

Modern reinforcement learning is Grasshopper, played on J

The whole method

1. Start with θ at random

The whole method

1. Start with θ at random
2. Play m episodes with π_θ

The whole method

1. Start with θ at random
2. Play m episodes with π_θ
3. Average their returns: that is $J(\theta)$

The whole method

1. Start with θ at random
2. Play m episodes with π_θ
3. Average their returns: that is $J(\theta)$
4. Step θ the way that raises it

The whole method

1. Start with θ at random
2. Play m episodes with π_θ
3. Average their returns: that is $J(\theta)$
4. Step θ the way that raises it
5. Repeat

The whole method

1. Start with θ at random
2. Play m episodes with π_θ
3. Average their returns: that is $J(\theta)$
4. Step θ the way that raises it
5. Repeat

$$\theta \leftarrow \theta + \alpha \nabla J(\theta)$$

The whole method

1. Start with θ at random
2. Play m episodes with π_θ
3. Average their returns: that is $J(\theta)$
4. Step θ the way that raises it
5. Repeat

$$\theta \leftarrow \theta + \alpha \nabla J(\theta)$$

a plus sign, not a minus: we are climbing

How the hill is climbed

$$J(\theta) = \mathbb{E}[G]$$

How the hill is climbed

$$J(\theta) = \mathbb{E}[G]$$

play some
episodes

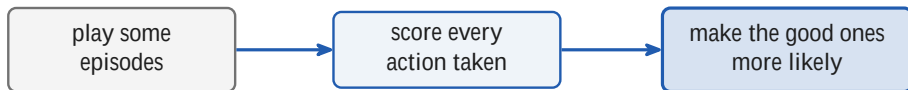
How the hill is climbed

$$J(\theta) = \mathbb{E}[G]$$



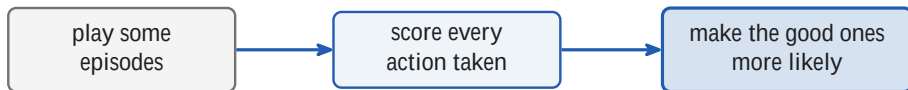
How the hill is climbed

$$J(\theta) = \mathbb{E}[G]$$



How the hill is climbed

$$J(\theta) = \mathbb{E}[G]$$



Everything below changes only the **middle box**

Three ways to score one action

	Score given to the action	In words
REINFORCE	G_t	the whole rest of the episode

Three ways to score one action

	Score given to the action	In words
REINFORCE	G_t	the whole rest of the episode
with a baseline	$G_t - V(s_t)$	better than this state usually goes

Three ways to score one action

	Score given to the action	In words
REINFORCE	G_t	the whole rest of the episode
with a baseline	$G_t - V(s_t)$	better than this state usually goes
Actor-critic	$r_t + \gamma V(s_{t+1}) - V(s_t)$	better than one step ahead predicted

Three ways to score one action

	Score given to the action	In words
REINFORCE	G_t	the whole rest of the episode
with a baseline	$G_t - V(s_t)$	better than this state usually goes
Actor-critic	$r_t + \gamma V(s_{t+1}) - V(s_t)$	better than one step ahead predicted

All three point the **same way** on average

Three ways to score one action

	Score given to the action	In words
REINFORCE	G_t	the whole rest of the episode
with a baseline	$G_t - V(s_t)$	better than this state usually goes
Actor-critic	$r_t + \gamma V(s_{t+1}) - V(s_t)$	better than one step ahead predicted

All three point the **same way** on average

they differ in how noisy one measurement is, and in how long you must wait for it



Section 7

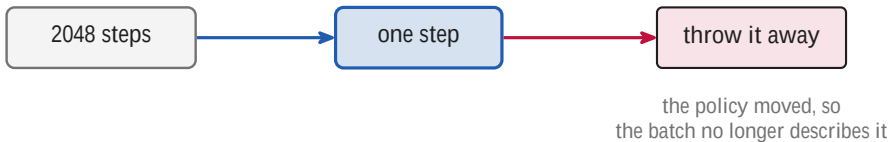
Proximal policy optimization

7

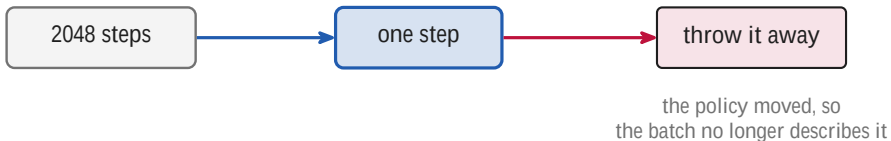
The data goes stale



The data goes stale

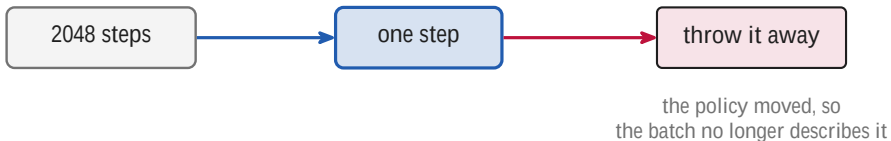


The data goes stale



Expensive. Those 2048 steps of π -ying bought **one** improvement.

The data goes stale



Expensive. Those 2048 steps of π -ying bought **one** improvement.

Can we use the batch more than once?

A stand-in for J

$$J(\theta)$$

what we actually want

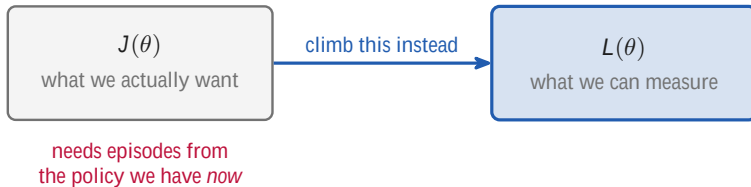
A stand-in for J

$$J(\theta)$$

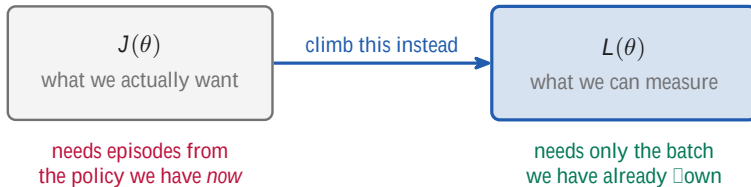
what we actually want

needs episodes from
the policy we have *now*

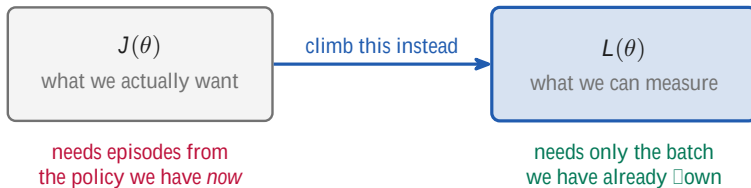
A stand-in for J



A stand-in for J

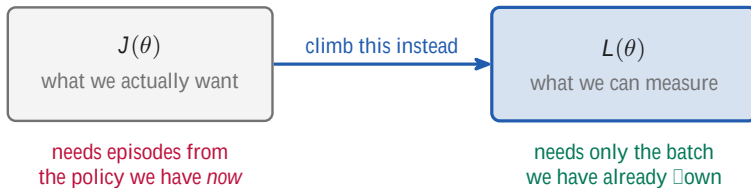


A stand-in for J



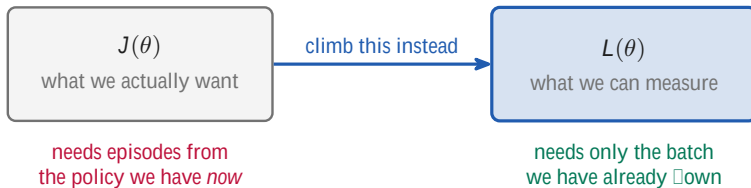
- Right here, L slopes exactly like J
- Over there, L can still be computed

A stand-in for J



- Right here, L slopes exactly like J
- Over there, L can still be computed

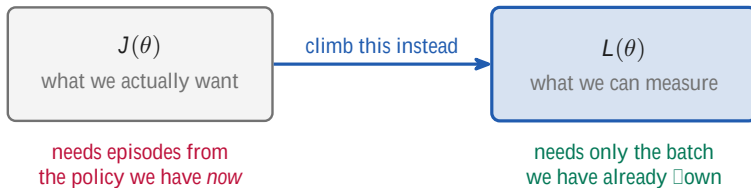
A stand-in for J



- Right here, L slopes exactly like J
- Over there, L can still be computed

the first three climbed the same J . This one does not.

A stand-in for J



- Right here, L slopes exactly like J
- Over there, L can still be computed

the first three climbed the same J . This one does not.

Climb the stand-in

How much has the policy moved

$$\rho_t = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

prob of choo...
 a_t at s_t

old prob of
choos...

$$\theta \leftarrow \theta + \alpha \bar{\nabla} J(\theta)$$

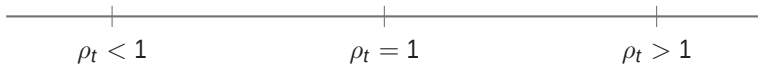
a_t at s_t

How much has the policy moved

$$\rho_t = \frac{\overbrace{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}^{\text{how likely now}}}{\underbrace{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}_{\text{how likely when we \texttt{new} it}}}$$

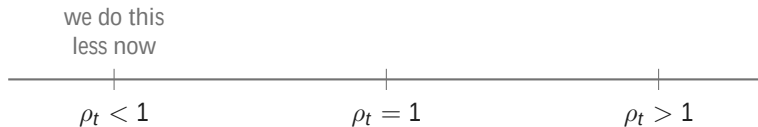
How much has the policy moved

$$\rho_t = \frac{\overbrace{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}^{\text{how likely now}}}{\underbrace{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}_{\text{how likely when we knew it}}}$$



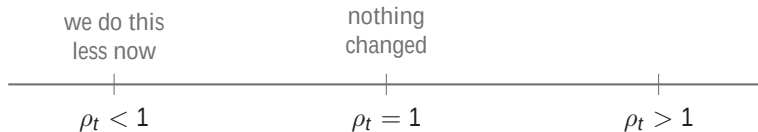
How much has the policy moved

$$\rho_t = \frac{\overbrace{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}^{\text{how likely now}}}{\underbrace{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}_{\text{how likely when we knew it}}}$$



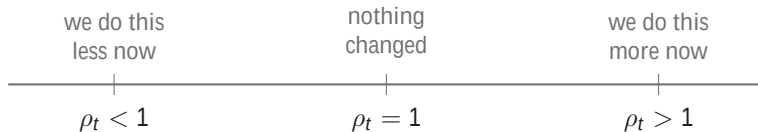
How much has the policy moved

$$\rho_t = \frac{\overbrace{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}^{\text{how likely now}}}{\underbrace{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}_{\text{how likely when we \texttt{new} it}}}$$



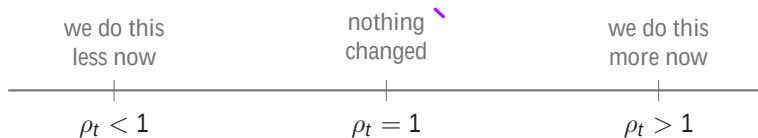
How much has the policy moved

$$\rho_t = \frac{\overbrace{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}^{\text{how likely now}}}{\underbrace{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}_{\text{how likely when we \texttt{new} it}}}$$



How much has the policy moved

$$\rho_t = \frac{\overbrace{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}^{\text{how likely now}}}{\underbrace{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}_{\text{how likely when we \texttt{new} it}}}$$

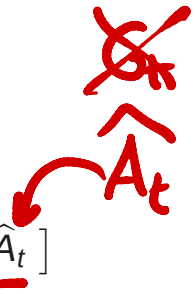


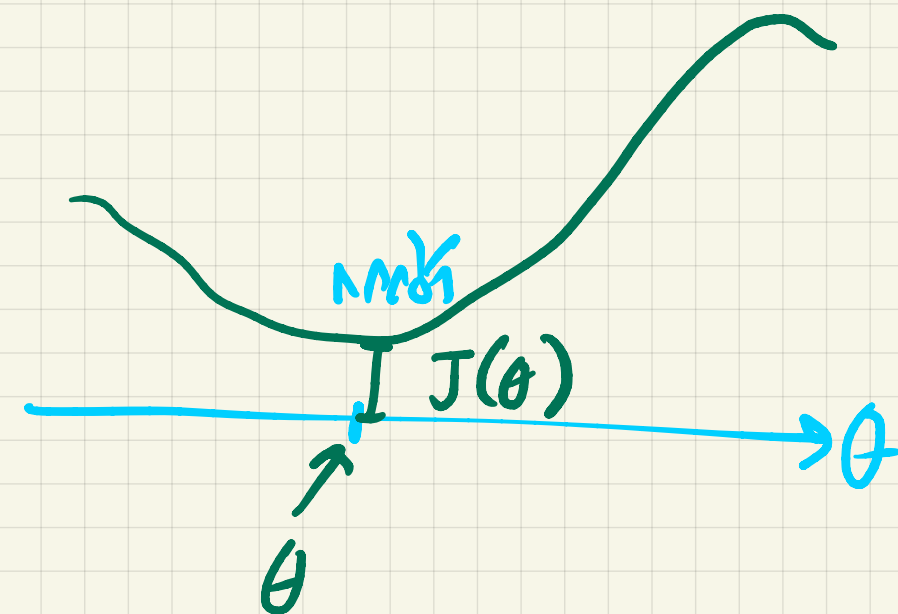
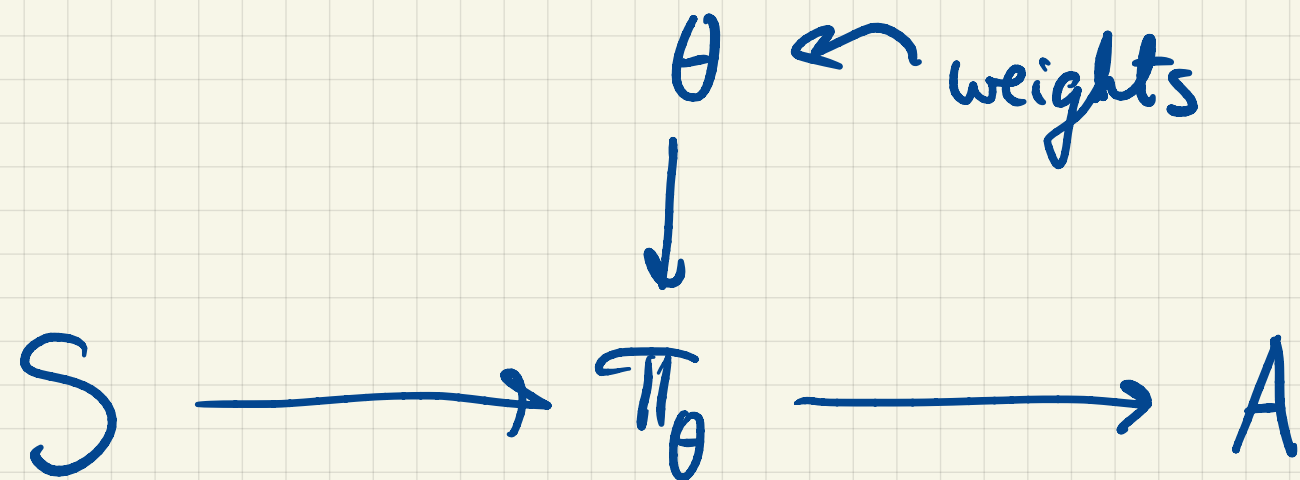
An old batch can still speak about the new policy

The stand-in, in one line

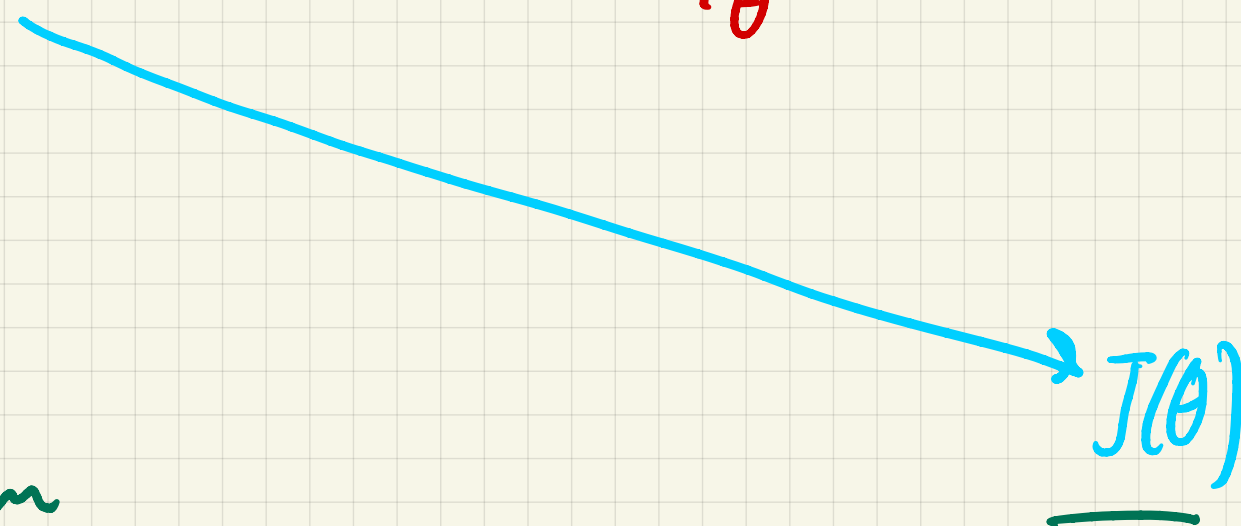
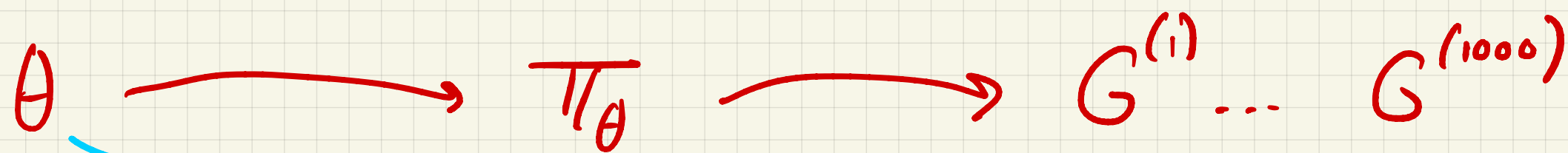
$L(\theta)$ = average over the batch we already knew

The stand-in, in one line

$$L(\theta) = \text{average} [\rho_t \hat{A}_t]$$




Grasshopper game



$$\text{avg} [G^{(1)} \dots G^{(1000)}] \approx E[G]$$

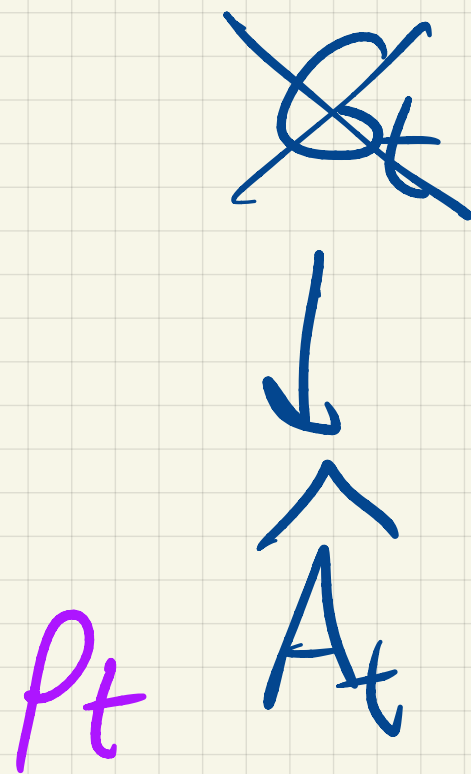
Friday Sam

~~A~~ → $\hat{A} = \dots$

PPG

→ $\hat{p} \hat{A}$

What does it mean to increase or reduce advantage?



The stand-in, in one line

$$L(\theta) = \text{average} \left[\underbrace{\rho_t}_{\text{how much likelier now}} \underbrace{\hat{A}_t}_{\text{how much better than expected}} \right]$$

The stand-in, in one line

$$L(\theta) = \text{average} \left[\underbrace{\rho_t}_{\text{how much likelier now}} \underbrace{\hat{A}_t}_{\text{how much better than expected}} \right]$$

at the start every $\rho_t = 1$
so L begins at zero: nothing claimed yet

The stand-in, in one line

$$L(\theta) = \text{average} \left[\underbrace{\rho_t}_{\text{how much likelier now}} \underbrace{\hat{A}_t}_{\text{how much better than expected}} \right]$$

at the start every $\rho_t = 1$
so L begins at zero: nothing claimed yet

and it slopes like J
so the first step is the honest one

The stand-in, in one line

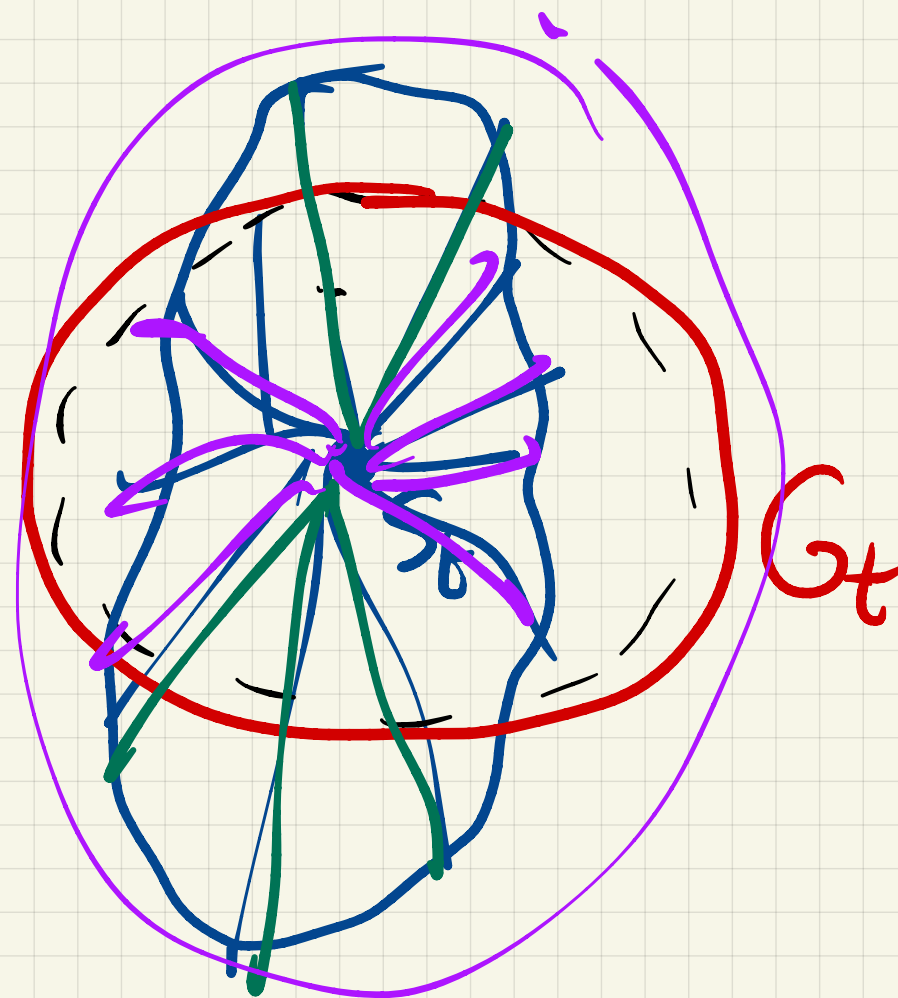
$$L(\theta) = \text{average} \left[\underbrace{\rho_t}_{\text{how much likelier now}} \underbrace{\hat{A}_t}_{\text{how much better than expected}} \right]$$

at the start every $\rho_t = 1$
so L begins at zero: nothing claimed yet

and it slopes like J
so the first step is the honest one

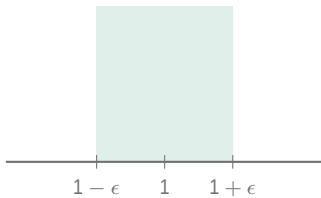
Zero where we stand, and the right way up

$[G_t]$

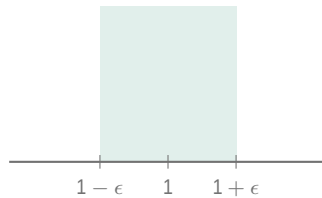


Do not walk too far

$\hat{A}_t > 0$, a good action

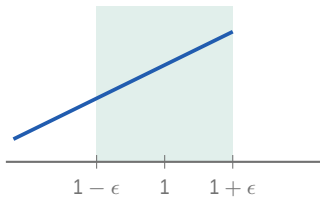


$\hat{A}_t < 0$, a bad action

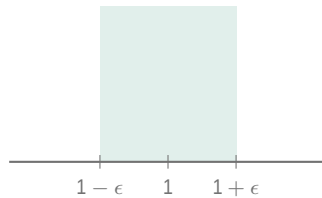


Do not walk too far

$\hat{A}_t > 0$, a good action

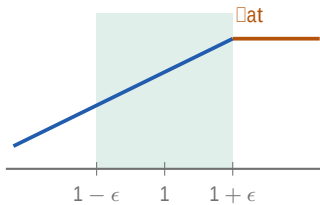


$\hat{A}_t < 0$, a bad action

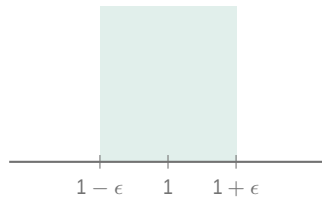


Do not walk too far

$\hat{A}_t > 0$, a good action

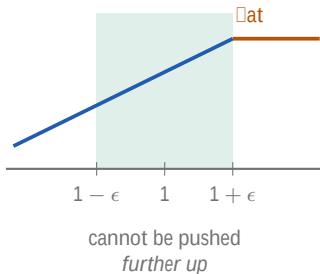


$\hat{A}_t < 0$, a bad action

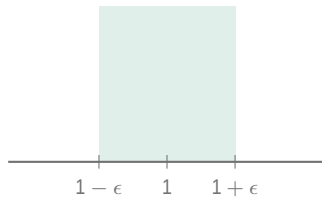


Do not walk too far

$\hat{A}_t > 0$, a good action

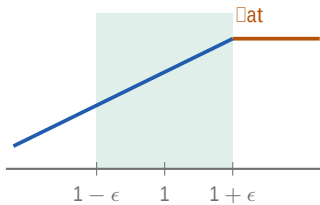


$\hat{A}_t < 0$, a bad action



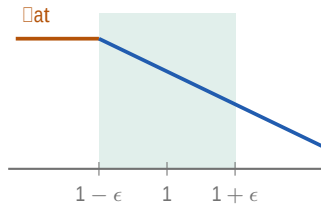
Do not walk too far

$\hat{A}_t > 0$, a good action



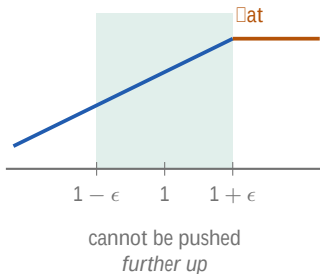
cannot be pushed
further up

$\hat{A}_t < 0$, a bad action

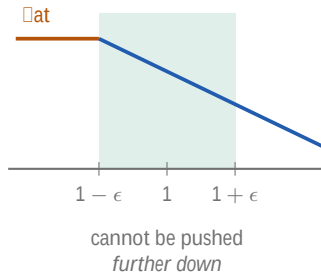


Do not walk too far

$\hat{A}_t > 0$, a good action

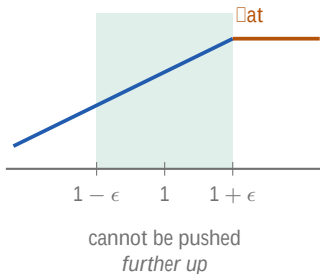


$\hat{A}_t < 0$, a bad action

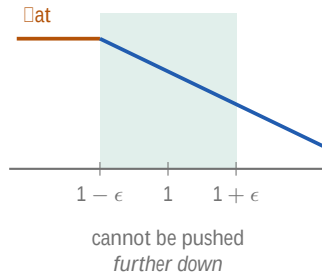


Do not walk too far

$\hat{A}_t > 0$, a good action



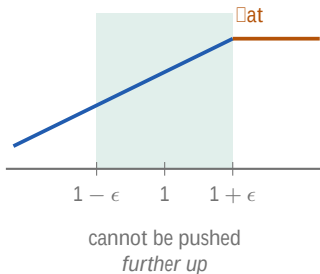
$\hat{A}_t < 0$, a bad action



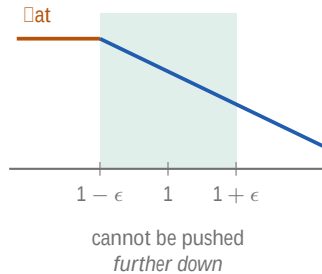
The π_{at} side is always the side that would **benefit** us

Do not walk too far

$\hat{A}_t > 0$, a good action



$\hat{A}_t < 0$, a bad action



The $\square_{at}$ side is always the side that would **⬇atter us**

the other side stays open, so a sample can always come back

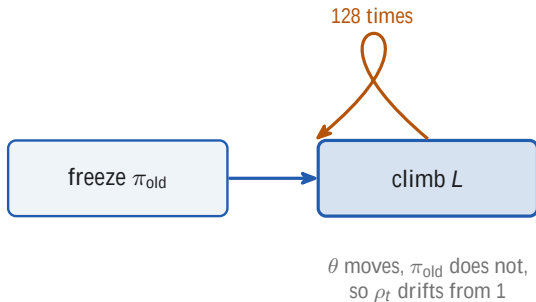
The same climb, many times

freeze π_{old}

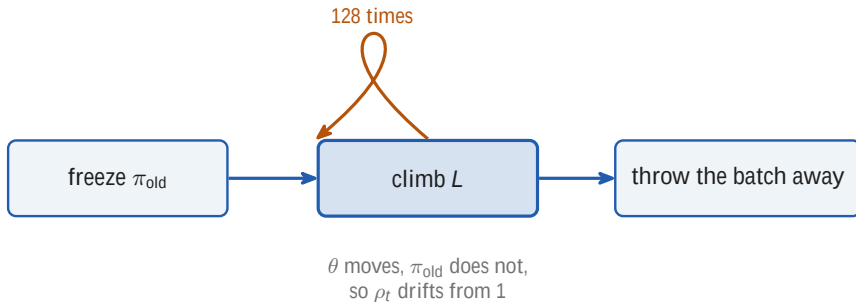
The same climb, many times



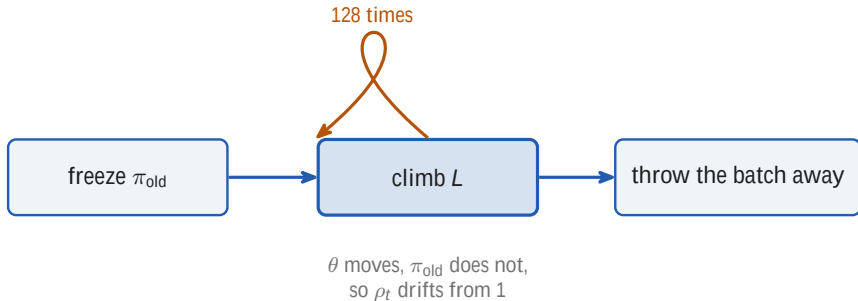
The same climb, many times



The same climb, many times

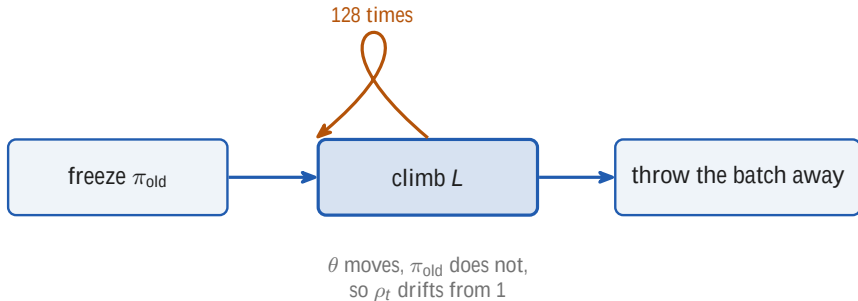


The same climb, many times



$$\theta \leftarrow \theta + \alpha \nabla L(\theta)$$

The same climb, many times



$$\theta \leftarrow \theta + \alpha \nabla L(\theta)$$

the same ascent as REINFORCE, on a different function

What this buys

REINFORCE | 1 improvement per batch

PPO

What this buys



What this buys

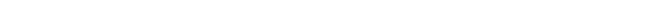
REINFORCE

1 improvement per batch

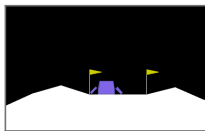
PPO

128

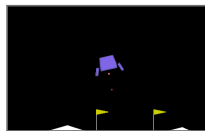
Same $\square$ ying. Same data. Same afternoon.

PPO  128

Same $\square$ ying. Same data. Same afternoon.



untrained



after the exercise